Cahier des charges – Projet de stage développement logiciel

Contexte du projet

L'entreprise utilise actuellement **ABSSolute** comme ERP. Certaines données de tournée existent déjà dans la base SQL liée à ABSSolute ou dans des vues SQL existantes. Aujourd'hui, les livreurs utilisent encore des fiches papier pour suivre leurs tournées et noter certaines informations terrain.

Le problème principal est que le fonctionnement actuel n'est pas assez pratique pour le **comptage des chariots, tapis ou éléments liés à la livraison**. L'objectif du projet est donc de créer une application mobile simple permettant aux livreurs de consulter leur tournée du jour, de saisir les informations nécessaires pendant la journée, puis de renvoyer les données en fin de tournée.

Le projet doit d'abord se concentrer sur un cas d'usage complet : **la prise en charge numérique d'une fiche de tournée quotidienne**. Les autres fonctionnalités, comme une interface Windows complète d'administration, peuvent être prévues dans l'architecture mais développées plus tard si le temps le permet.

Objectif du stage

Le stagiaire aura pour mission de concevoir et développer une solution complète comprenant :

Application mobile (Android 12)

L'application mobile sera utilisée par les livreurs. Elle sera développée avec **Visual Studio**, probablement en **.NET MAUI** ou avec une autre solution compatible avec Android.

Elle devra permettre de :

- charger la tournée du jour ;
- afficher les clients ou points de livraison ;
- consulter les informations utiles à la livraison ;
- saisir les données terrain ;
- stocker localement les données pendant la journée ;
- envoyer les données en fin de tournée.

API intermédiaire ASP.NET Core

L'API intermédiaire sera développée en **ASP.NET Core**. Elle servira de point de communication entre l'application mobile et SQL Server.

Elle jouera un rôle central dans le projet, car elle permettra de :

- éviter une connexion directe entre le téléphone et SQL Server ;
- lire les données nécessaires via des vues SQL ;
- envoyer au mobile uniquement les données utiles ;
- recevoir les saisies effectuées par les livreurs ;
- contrôler les données reçues ;
- transformer les données JSON en données SQL ;
- écrire les résultats dans des tables dédiées ;
- faciliter l'évolution du projet.

Des procédures stockées pourront éventuellement être utilisées plus tard si l'entreprise souhaite centraliser certaines règles métier directement côté SQL Server.

SQL Server

SQL Server sera utilisé pour :

- lire les données issues d'ABSSolute via des vues SQL ;
- stocker les données saisies par les livreurs ;
- conserver les informations de synchronisation ;
- préparer l'exploitation administrative des données.

Les données mobiles ne devront pas modifier directement les tables ABSSolute. L'approche recommandée est d'utiliser des **tables dédiées au projet mobile**.

Exports et interface administrative

Dans un premier temps, des exports **CSV ou JSON** sont prévus afin de permettre à l'administration de consulter ou exploiter les données récoltées.

Une interface administrative Windows pourra être développée plus tard si le temps le permet. Elle permettrait de consulter les tournées, suivre les validations, gérer certaines données et afficher les résultats de manière plus confortable.

Fonctionnement actuel

Aujourd'hui :

1. Les données de tournée existent dans ABSSolute ou dans une base SQL server liée à ABSSolute.
2. Les fiches de tournée sont utilisées sous forme papier ou Excel.
3. Les livreurs suivent leur tournée manuellement.
4. Les informations de terrain, comme le nombre de chariots, doivent être notées ou recomptées.
5. L'administration doit ensuite récupérer ou retraiter les informations.

Fonctionnement attendu avec l'application

Début de journée

Le livreur ouvre l'application mobile au dépôt.

Il s'identifie dans un premier temps par son nom, puis choisit ou confirme sa tournée. L'application appelle ensuite l'API pour récupérer les données utiles de la tournée du jour.

La tournée est ensuite enregistrée localement sur le téléphone afin que le livreur puisse travailler pendant la journée sans dépendre d'une connexion permanente.

Pendant la tournée

Le livreur consulte la liste des clients ou points de livraison à traiter.

Pour chaque client, il peut voir :

- le numéro client ;
- le nom client ;
- le point de livraison ;
- la description du point de livraison ;
- la zone ;
- la tournée ;
- les instructions ;
- le type de linge ou vêtement ;
- les informations de retour si nécessaire.

Le livreur peut ensuite saisir les informations terrain, notamment :

- le nombre de chariots livrés ;
- le nombre de chariots repris ou retournés ;
- un commentaire en cas particulier ;
- le statut du passage ;
- la validation du passage.

Les données sont conservées localement sur le téléphone jusqu'à l'envoi final.

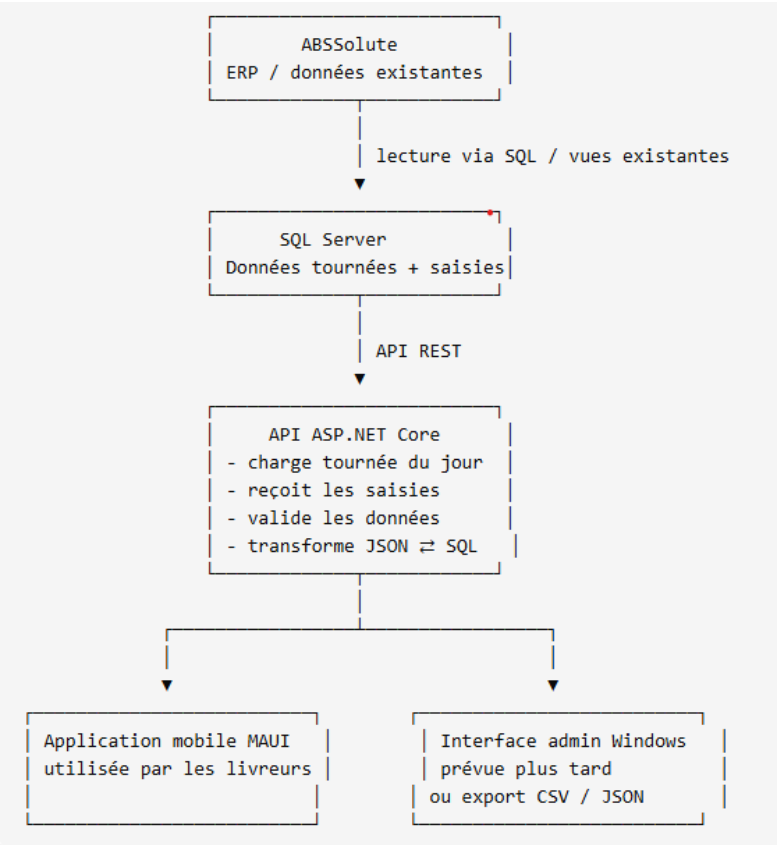
Fin de journée

En fin de journée, le livreur appuie sur un bouton d'envoi.

L'application envoie les données en JSON à l'API. L'API vérifie les données, évite les doublons, puis insère les résultats dans SQL Server.

Après synchronisation réussie, les données ne sont plus modifiables côté livreur.

Schéma de l'architecture globale :



Cette architecture est adaptée parce qu'elle sépare clairement les responsabilités.

L'application mobile se concentre sur l'usage terrain : consulter, saisir et envoyer.

L'API centralise la logique métier : contrôle, transformation, sécurité, communication avec SQL Server et gestion de la synchronisation.

SQL Server conserve les données proprement.

Les exports ou l'interface Windows peuvent être ajoutés plus tard sans modifier toute l'application mobile.

Données utilisées

Données récupérées depuis ABSSolute / SQL Server

Les données de départ correspondent aux informations de tournée déjà existantes.

Champ	Rôle dans l'application

NUM_CLI	Identifiant du client
NOM_CLI	Nom du client
PDL	Point de livraison
PDL_DESC	Description du point de livraison
JOUR_TOURNEE	Jour de tournée
SCHEMA_LIV	Schéma de livraison
TOURNEE	Code tournée
TOURNEE_DESC	Nom de la tournée, ne détermine pas l'ordre réel
ARRET	Ordre des clients, croissant
JOUR_TOURNEE_RETOUR	Jour de tournée retour
TOURNEE_RET	Code tournée retour
TOURNEE_RETOUR_DESC	Description du retour
INSTRUCTIONS	Instructions particulières pour le client

`ZONE_DECH`	Zone de déchargement
`DATEINACTIVE`	déjà un filtre donc on traite pas
`Zone`	Zone ou regroupement logistique

Ces données servent principalement à **préremplir l'application mobile** pour éviter que le livreur ne ressaisisse des informations déjà connues.

Données saisies sur mobile par le livreur :

Donnée	Statut précisé
Nombre de chariots livrés	Saisi par le livreur avec boutons `+` et `-`
Nombre de chariots repris ou retournés	saisi par le livreur afin d'éviter les oublis.
Type de vêtements / linge	Prérempli et non modifiable par le livreur
Commentaire livreur	Saisie possible pour cas particulier
Statut du passage	Fait / non fait / anomalie
Heure de validation	Enregistrée automatiquement
Identifiant du livreur	À associer à chaque validation

Tournée choisie	Associée au livreur au début de journée

Le but n'est pas de créer une application trop lourde. Il faut que le livreur puisse saisir rapidement les informations, avec des boutons simples comme `+`, `-`, `Valider`, `Commentaire`. Les quantités négatives sont interdites. Les retours doivent être gérés dans un champ séparé, et non par une valeur négative.

Utilisateurs et rôles

Livreurs

Environ **une vingtaine de livreurs** utiliseront l'application.

Chaque livreur est normalement affecté à une tournée. Cependant, une tournée peut être reprise par un autre livreur en cas d'absence, maladie ou remplacement.

Le livreur s'identifie

↓

Il choisit ou confirme sa tournée

↓

Il charge la tournée du jour

↓

Il saisit les données client par client

↓

Il envoie la tournée en fin de journée

Administrateur

L'administrateur doit pouvoir :

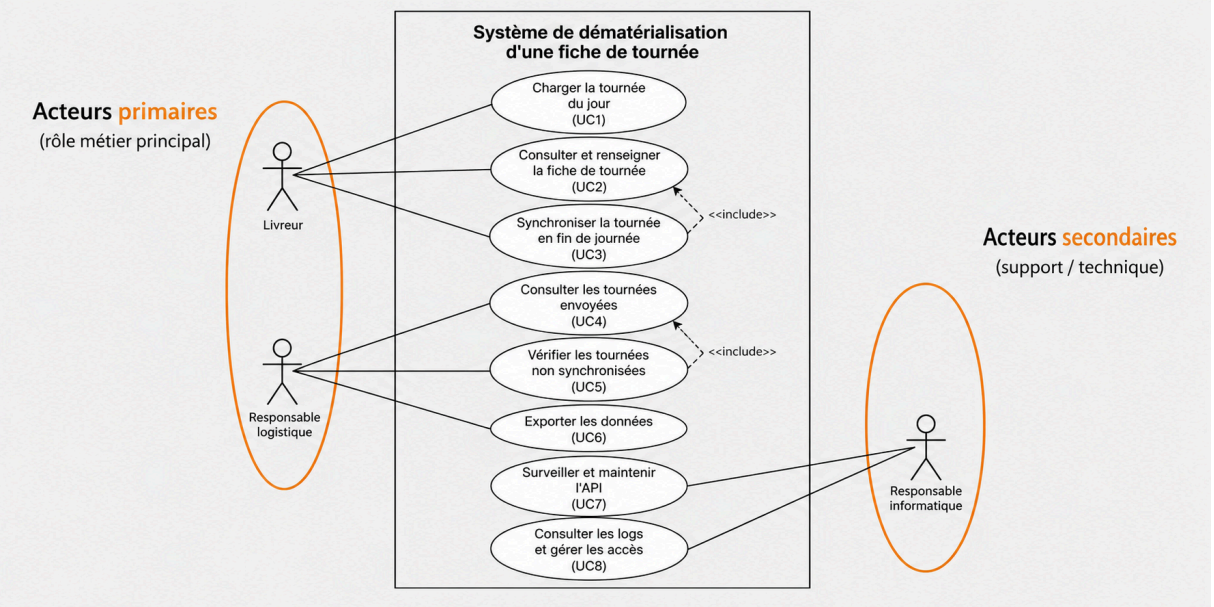
- gérer ou modifier les clients dans les tournées ;
- gérer les instructions ;
- consulter les données envoyées ;
- vérifier si toutes les tournées ont été envoyées ;
- relancer manuellement les livreurs si une tournée n'a pas été validée ;
- exporter les données au format CSV ou JSON ;

- éventuellement accéder plus tard à une interface Windows.

Responsable informatique

Le responsable informatique de l'entreprise, ton maître de stage, devra pouvoir :

- maintenir l'API ;
- la redémarrer si nécessaire ;
- surveiller son fonctionnement ;
- gérer les accès SQL ;
- faire évoluer le projet après le stage.



Règles métier

Validation côté livreur

Pour la première version, il n'est pas prévu d'imposer beaucoup de champs obligatoires au livreur afin de ne pas complexifier l'usage terrain.

Les données saisies devront rester rapides à renseigner. Les contrôles principaux porteront surtout sur :

- l'impossibilité de saisir des quantités négatives ;
- l'enregistrement automatique de l'heure de validation ;
- l'association de la saisie au livreur ;
- l'association de la saisie à la tournée ;
- la gestion du statut du passage.

Il faut rendre obligatoire un commentaire en cas de statut `anomalie` ou `non fait`.

Validation côté administrateur

Pour la partie administration, les champs nécessaires à la création ou modification d'un client dans une tournée sont :

- numéro client ;
- nom client ;
- commentaire ou instruction ;
- numéro PDL ;
- nom PDL ;
- zone ;
- tournée.

Ces règles pourront être précisées lors de la réalisation de l'interface administrative ou des exports.

Instructions

Les instructions temporaires sont liées à un client.

Elles sont préparées avant le début de la tournée. Le livreur peut les consulter, mais ne peut pas les modifier ni en ajouter dans la première version.

Clients fermés

Les clients fermés pourront être gérés par un filtre côté application.

Deux affichages seront possibles :

- masquer les clients fermés ;
- afficher les clients fermés avec un statut spécifique.

Cette solution permet de laisser de la souplesse à l'administration et aux utilisateurs sans supprimer définitivement l'information de la tournée.

Synchronisation et stockage local

L'application mobile devra stocker les données localement pendant la journée, par exemple avec une base locale SQLite.

Cela permet au livreur de continuer à utiliser l'application même si la connexion réseau n'est pas disponible pendant la tournée.

Statuts de synchronisation

L'application devra prévoir des statuts simples :

- non envoyé ;
- envoyé ;
- erreur d'envoi.

Échec d'envoi

Si l’envoi échoue le soir, une vérification manuelle sera effectuée par le responsable logistique.

Au début du projet, les feuilles papier seront conservées en parallèle afin d’éviter les pertes d’information pendant la phase de transition.

L’application ne permettra pas le renvoi d’une tournée le lendemain afin d’éviter les conflits de données. Dans le fonctionnement actuel de l’entreprise, la validation est vérifiée chaque soir par le responsable logistique.

Double envoi

La protection contre les doubles envois devra être gérée côté API.

L’API devra vérifier si une tournée du même chauffeur, à la même date, avec le même code tournée, a déjà été envoyée.

Exemple de comportement attendu :

Si la tournée a déjà été envoyée :

API → "Tournée déjà synchronisée"

Mobile → "Déjà envoyée le 27/04 à 16h12"

Modification après synchronisation

Après synchronisation réussie, la tournée sera bloquée côté livreur.

Une tournée envoyée ne pourra pas être corrigée par le livreur. Les corrections administratives ne sont pas prévues dans la première version.

Avant l’envoi final, l’application devra afficher un message de confirmation indiquant que les données ne seront plus modifiables après synchronisation.

Hébergement de l’API :

L’API sera hébergée dans une **VM Windows** située sur le même serveur physique que SQL Server, si les ressources du serveur le permettent.

Cette solution permet :

- d’isoler l’API de SQL Server ;
- d’éviter d’installer directement l’application sur le serveur SQL ;
- de redémarrer l’API sans toucher directement à SQL Server ;
- de rester dans l’infrastructure interne de l’entreprise ;
- de faciliter la maintenance ;
- de limiter les coûts supplémentaires.

Mode d’exécution de l’API

Le choix recommandé est d’héberger l’API ASP.NET Core sous **IIS** dans la VM Windows.

IIS n’est pas open source, mais il est inclus avec Windows Server. Il ne devrait donc pas générer de coût supplémentaire si l’entreprise dispose déjà d’une licence Windows Server.

Le développement pourra se faire localement avec Visual Studio ou en console, mais la version exploitable par l’entreprise devra être déployée sous IIS.

Éléments à prévoir :

- VM Windows ;
- IIS activé ;
- .NET Hosting Bundle installé ;
- API ASP.NET Core publiée ;
- configuration du site ou de l’application dans IIS ;
- accès SQL Server configuré ;
- logs de fonctionnement ;
- procédure de redémarrage documentée.

Accès réseau à l’API

La solution la plus adapté à nos besoin est un accès à l’API via le **Wi-Fi de l’entreprise au dépôt** :

Le matin :

téléphone connecté au Wi-Fi du dépôt

→ chargement de la tournée

Pendant la journée :

utilisation hors connexion

→ stockage local sur le téléphone

Le soir :

téléphone connecté au Wi-Fi du dépôt

→ envoi des données vers l’API

Avantage :

- elle évite d’exposer l’API sur Internet ;
- elle est plus simple à sécuriser ;

- elle correspond au fonctionnement prévu du projet ;
- elle évite de dépendre de la 4G/5G pendant la tournée ;
- elle limite la complexité réseau pour une première version.

Point à vérifier avant déploiement : les téléphones professionnels doivent pouvoir se connecter au Wi-Fi de l'entreprise au dépôt. Le wifi fonctionne bien dans le dépôt.

HTTPS

Pour la version finale exploitable en entreprise, l'API devra utiliser HTTPS.

SQL Server et stockage des données

Accès aux données :

L'accès aux données ABSSolute se fera via des **vues SQL**, cette approche permet de :

- ne pas dépendre directement des tables internes ABSSolute ;
- limiter les risques de modification accidentelle ;
- simplifier les données envoyées à l'application mobile ;
- centraliser les filtres nécessaires ;
- rendre l'API plus stable.

Lecture des vues ABSSolute Serveur : SRVSQL\SQLEXPRESS,58734 Base : lavinprosli

Tables dédiées au projet mobile

Les données saisies par les livreurs seront stockées dans SQL Server dans des tables dédiées au projet mobile. L'API utilisera un compte SQL dédié avec des droits limités : lecture seule sur les vues issues d'ABSSolute et écriture uniquement sur les tables du projet mobile.

L'entreprise dispose d'un environnement de test. Le développement se fera dans cet environnement. Une fois le projet terminé et validé, l'environnement pourra être nettoyé puis devenir l'environnement final.

Base de test projet mobile Serveur : SRVSQL\SQL2017 Base : bd_eric

Sauvegarde

Des sauvegardes sont déjà prévues dans l'environnement SQL Server.

Le rythme indiqué est de plusieurs sauvegardes par jour, dont une sauvegarde toutes les demi-heures.

Le projet devra respecter ce fonctionnement existant.

Log

L'API devra journaliser les événements importants : chargement d'une tournée, réception d'une synchronisation, erreur d'envoi, tentative de double envoi et erreurs SQL éventuelles.

Exigences de développement

Qualité du code

Le code devra respecter les principes suivants :

- nommage clair et cohérent ;
- séparation des responsabilités ;
- découpage propre entre interface, logique métier et accès aux données ;
- commentaires utiles ;
- respect des bonnes pratiques ;
- gestion claire des erreurs ;
- structure évolutive.

Les commentaires devront expliquer les choix techniques ou métier, et non décrire des évidences.

Réutilisabilité

Le système devra être conçu pour pouvoir :

- être adapté à d'autres cas d'usage proches ;
- être maintenu facilement ;
- permettre l'ajout de nouveaux champs sans refonte complète ;
- isoler autant que possible la logique métier spécifique ;
- réutiliser la même base technique pour d'autres formulaires mobiles.

Documentation

- un README technique ;
- un schéma d'architecture global ;
- un schéma de base de données ;
- une documentation des routes API ;
- une documentation Swagger/OpenAPI ;
- une explication simple du déploiement ;
- une procédure de redémarrage ou de maintenance de l'API ;
- des exemples de requêtes JSON ;
- une description du fonctionnement de la synchronisation.

Tests automatique (optionnel)

Des tests automatiques seront prévus lorsque le premier prototype fonctionnera.

Les tests devront prioritairement couvrir :

- les routes principales de l'API ;
- la validation des données reçues ;
- la protection contre les doubles envois ;
- l'insertion SQL ;
- les cas d'erreur de synchronisation.

Conclusion :

L'architecture retenue repose sur une application mobile Android, une API ASP.NET Core hébergée dans une VM Windows sous IIS, et SQL Server comme base centrale.

Le choix prioritaire est de faire fonctionner l'application en mode principalement hors ligne pendant la tournée, avec chargement et envoi des données depuis le dépôt via le Wi-Fi interne de l'entreprise.

Cette approche permet de répondre au besoin métier sans complexifier inutilement le projet, tout en gardant une base évolutive pour de futurs usages.